AI Engineer Project: LLMOps + Agent Evals + Auto-Routing

One project that combines all three. By @ajays_vision_

Basic RAG chatbots and MNIST classifiers are on every fresher resume. Build one system that uses the three skills AI teams actually hire for, and prove it works with real numbers.

The 3 topics, in simple terms

Think of a restaurant: a junior cook (cheap model), a head chef (expensive model), a manager, and a taster.

Topic	What it means	Restaurant version
Auto-routing	Send each request to the right model: easy tasks to a cheap, fast model, hard tasks to a strong one.	*Manager decides who cooks each order*
LLMOps	Log every request (model, tokens, latency, cost), cache repeats, fall back when a provider fails.	*Manager's order book and backup cook*
Agent evals	Test whether an AI agent did the task right: the final answer, the tools it used, the steps it took.	*The taster checking every dish*

The project: one smart AI gateway

A backend service that sits between apps and AI models. It's infrastructure, not a chatbot or an app.

1. An app sends a question to your gateway instead of calling the AI directly.
2. The **router** checks the task: easy → cheap model, hard → strong model.
3. The model answers. If the provider fails, the gateway retries, then **falls back** to another model.
4. The gateway **logs** one row: model, tokens, latency, cost, success or error.
5. A small **agent** with 2–3 tools takes a fixed exam of 20–50 tasks through the gateway, and the **evals** score it.

The point: run the same exam with always-cheap, always-strong and your router. Compare cost and pass rate.

Tech stack (free to start)

API: Python, FastAPI (async), httpx ▪ **Models:** Groq free tier, or any OpenAI-compatible API ▪ **Storage:** SQLite, then PostgreSQL ▪ **Cache:** in-memory, then Redis ▪ **Shipping:** Docker, GitHub Actions, AWS (new-account credits; set a zero-spend budget alert first). Keep API keys in a `.env` file listed in `.gitignore`, never in code.

Build plan: 5 phases, about 2–3 weeks

1. **Gateway:** one `POST /v1/chat` endpoint, every request logged with tokens, latency and cost. Tests with a mocked provider.
2. **Reliability:** second model, retries with backoff, automatic fallback, response cache.
3. **Router:** start with rules (prompt length, keywords, code present). Log the route chosen and why.
4. **Agent + evals:** small agent with tools (calculator, lookup), 20–50 tasks with known answers. Score answer, tools used, steps.
5. **Prove and ship:** run the exam under 3 policies, chart cost vs pass rate, README, Docker, deploy, evals in CI.

Use AI coding tools to go faster, but write the router and scorers yourself. Interviewers will ask you to explain them.

What goes on your resume

Fill X and Y from your own eval runs, never guesses:

Built an LLM gateway with auto-routing, fallback and cost tracking; auto-routing cut LLM cost by X% while keeping agent eval pass rate within Y points of always using the strongest model.

Add a live link, a GitHub repo with an architecture diagram, and a README that explains your design decisions.

Interview questions to prepare

- How does your router decide which model gets a request?
- How do you evaluate an agent, not just one LLM answer?
- How do you calculate the cost of an LLM call? What happens when a provider times out or rate-limits you?
- Why not just use LiteLLM or Portkey? *(You built the core to understand it; your focus is evals that prove routing is safe.)*